

Building Quantum Networks

Prof. Christiancarmine Esposito
esposito@unisa.it

Maria Caterina D'Aloia
mdaloia@unisa.it

What We'll Cover Today

01

...

Quantum Network Elements

The physical building blocks of any quantum network: qubits, entanglement, channels, memories, repeaters, and heralding.

02

...

The NetSquid Framework

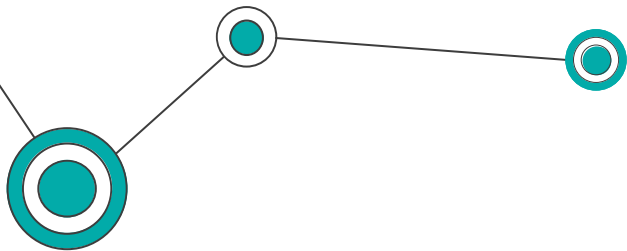
Discrete-event simulation architecture, component model, quantum state formalisms, protocol programming, and installation.

03

...

Code Walkthrough

Reading a complete NetSquid simulation — source, processor, channels, wiring, BSM, heralding protocols.

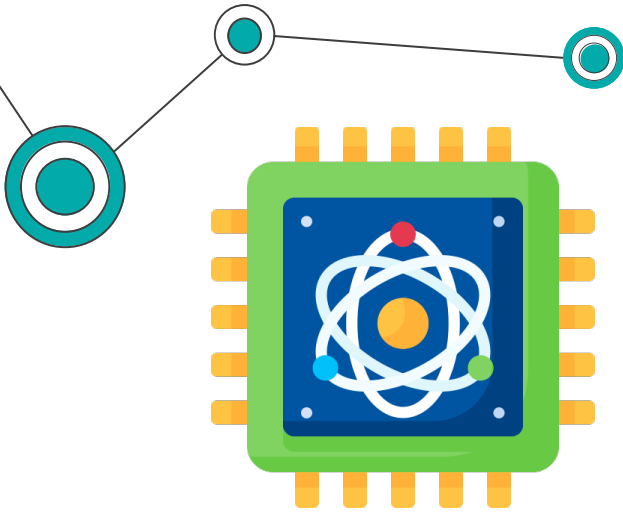


PART I

Quantum Network Elements

The physics you need to build a network

Why Quantum Networks?

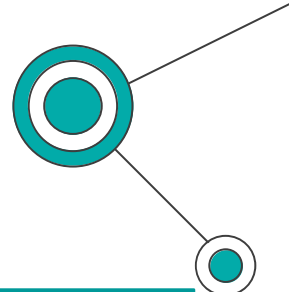
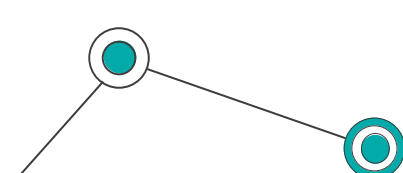


Classical Networks Can:

- Transfer bits reliably at global scale
- Copy, route, and amplify signals freely
- Enable the internet as we know it
- Store data indefinitely without degradation

Quantum Networks Add:

- Provably secure key distribution (QKD)
- Distributed quantum computation
- Blind quantum computing
- Long-baseline quantum sensing & metrology
- Quantum internet applications not yet invented



The No-Cloning Theorem – Why Networks Work Differently

An unknown quantum state **CANNOT** be perfectly copied.

Classical relay model:

Signal arrives → amplify → forward. Works because you can copy bits freely.

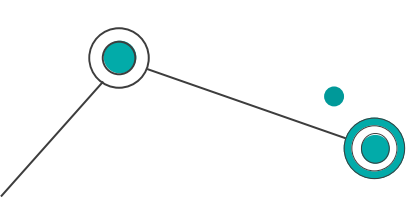
Quantum relay model:

Qubit arrives → copying is FORBIDDEN by physics → you need a different mechanism.

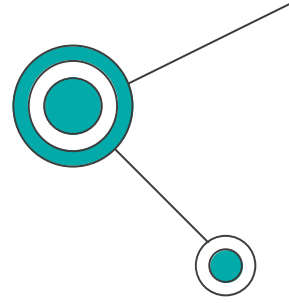
Consequence:

Direct quantum links lose signal exponentially with distance (0.2 dB/km in fiber).

→ **We need the quantum repeater**



The Qubit & Entanglement



THE QUBIT

Two-level quantum system: $|0\rangle$, $|1\rangle$, or any superposition $\alpha|0\rangle + \beta|1\rangle$.

Physical realizations: electron spins (NV centers), photons, trapped ions, superconducting circuits.

Measuring a qubit collapses its state — irreversible!

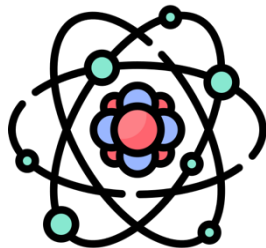
ENTANGLEMENT

Two qubits are entangled when their joint state cannot be written as a product of individual states.

Bell state: $|\Phi^+\rangle = (|00\rangle + |11\rangle) / \sqrt{2}$

Measuring one qubit instantly determines the other — regardless of distance. Entanglement is the channel of quantum networking.

5 Key Components of a Quantum Network



① Quantum Source

Generates entangled pairs (Bell states) on demand or on a clock trigger. In NetSquid: QSource.

② Quantum Channel

Transmits qubits via optical fiber. Subject to loss (FibreLossModel) and depolarizing noise.

③ Quantum Memory

Stores qubits while waiting for operations. Coherence time T_2 is the critical hardware parameter.

④ Quantum Processor

Applies gates (X, Z, H, CNOT) and measurements. Executes quantum programs. In NetSquid: QuantumProcessor.

⑤ Classical Channel

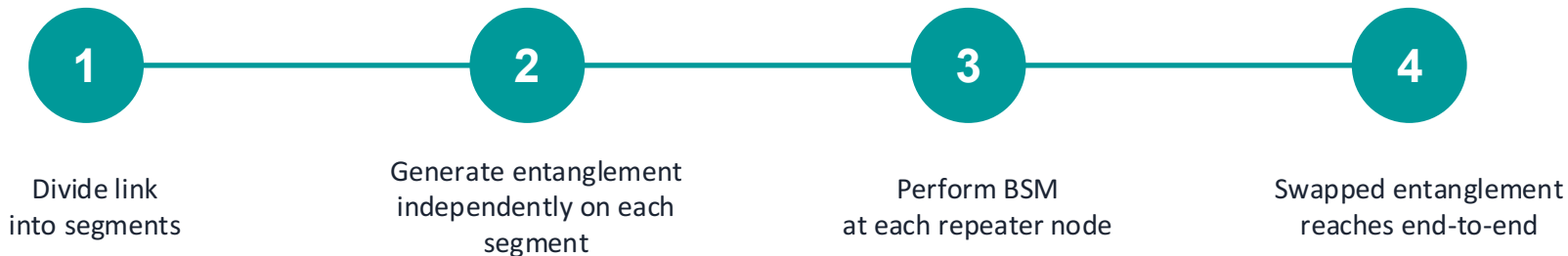
Carries heralding messages and measurement outcomes. Enables protocol coordination between nodes.

The Quantum Repeater

Problem: Optical fiber attenuates photons exponentially. At 0.2 dB/km, a 1000 km link loses essentially all signal.

Classical amplifier: copy and re-send → FORBIDDEN by no-cloning.

Quantum repeater: divide the link into shorter segments, use entanglement swapping at each hop.



Result: end-to-end entanglement across distances no direct link could achieve.

Entanglement Swapping

Step 1

Local generation

Alice generates Bell pair (A_{LOCALE}, A_{REMOTO})
Bob generates Bell pair (B_{LOCALE}, B_{REMOTO})

Step 2

Transmission

A_{REMOTO} travels via quantum channel $\rightarrow$ Repeater
 B_{REMOTO} travels via quantum channel $\rightarrow$ Repeater

Step 3

BSM at Repeater

Repeater performs Bell State Measurement on (A_{REMOTO}, B_{REMOTO})
This projects $A_{LOCALE} \otimes B_{LOCALE}$ into an entangled state

Step 4

Classical Heralding

Repeater sends BSM result to Alice and Bob
Alice & Bob can now exploit the $A_{LOCALE} \leftrightarrow B_{LOCALE}$ correlation

Key insight: No qubit from Alice ever reaches Bob directly. Entanglement is swapped, not transmitted.

Heralding & Noise

HERALDING

A classical message sent after a quantum event to confirm success or failure.

Why it matters:

- BSMs succeed with probability < 1
- Alice & Bob need confirmation
- Heralding latency interacts with memory coherence time

In our code:

```
{type, round, bell_index, time}  
sent over classical fiber channel
```

NOISE & DECOHERENCE

Depolarizing noise

Qubit randomly flips — random X,Y,Z errors with probability p .

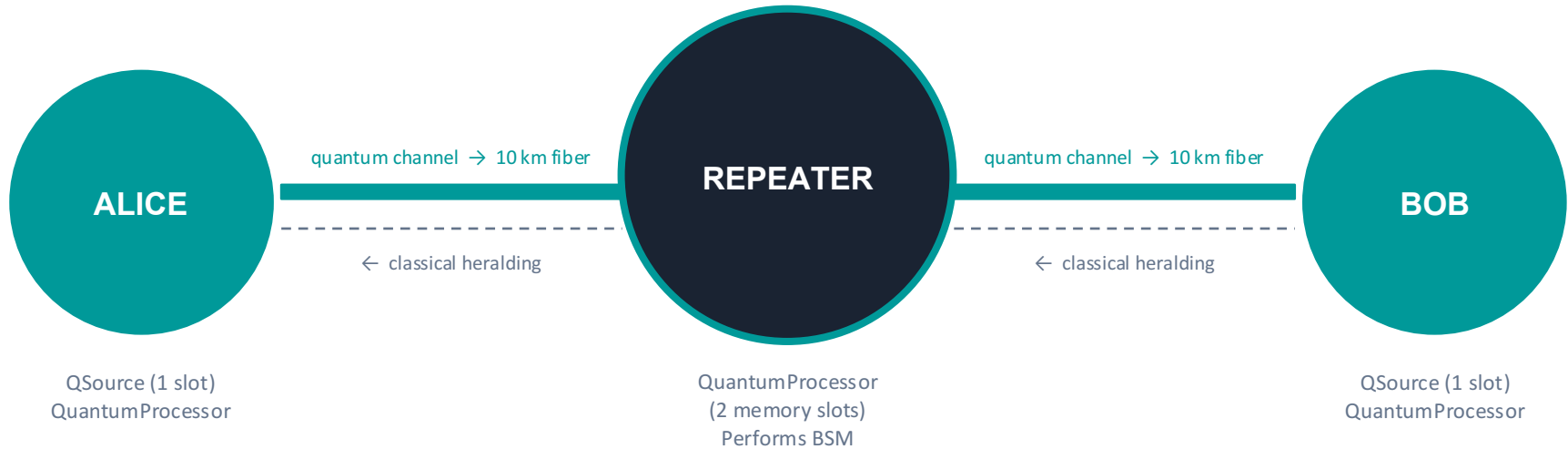
Dephasing (T_2 noise)

Phase of a superposition decays over time — coherence time T_2 is critical.

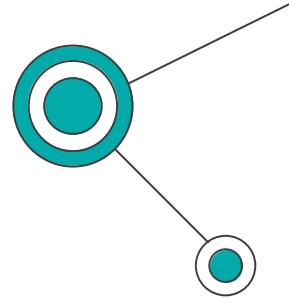
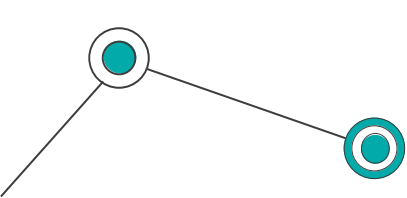
Photon loss

Qubits in photons disappear in fiber — probability grows exponentially with distance.

Our Target Topology



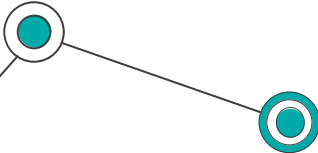
This is exactly the topology we simulate in Part III. Each arrow carries a different type of signal — quantum or classical.



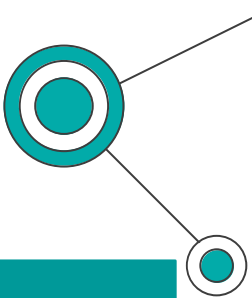
PART II

NetSquid Framework

Discrete-event simulation for quantum networks



What Is NetSquid?



NETwork Simulator for QUantum Information using Discrete events

Developed at: QuTech — Delft University of Technology & TNO (Coopmans et al., arXiv:2010.12535, 2021)

What it does: A Python simulation platform for quantum networks and modular quantum computing — from the physical layer to the application level.

ACCURACY

Models time-dependent noise, decoherence, hardware non-idealities with physical fidelity.

MODULARITY

Stack protocols and components to build complex networks from simple, reusable parts.

SCALABILITY

Demonstrated on repeater chains of up to 1,000 nodes.

Why Discrete-Event Simulation?

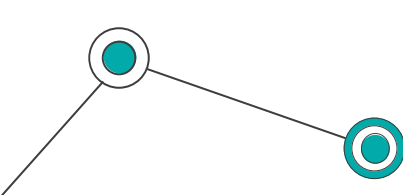
Key insight: In a quantum network, time does not flow continuously — it jumps between events.

What counts as an event?

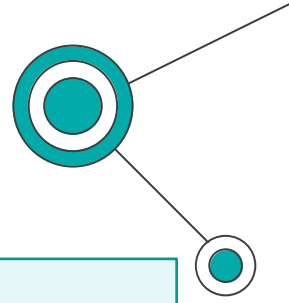
- 1 A qubit arrives at a memory port
- 2 A photon is detected at a midpoint station
- 3 A classical heralding message is received
- 4 A timer expires (source trigger interval)



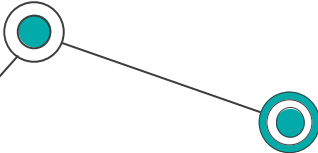
Benefit: Quantum decoherence is applied retroactively when a qubit is accessed — no continuous time integration needed. The engine jumps directly to the next event.



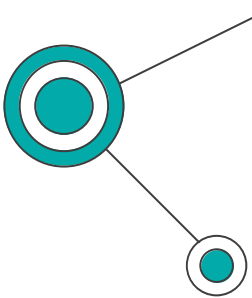
NetSquid Architecture



protocols	NodeProtocol · ServiceProtocol · LocalProtocol
nodes	Node · Network · Connection
components + models	Component · Port · Model · DelayModel · ErrorModel
qubits	Qubit · QState · KET / DM / STAB / GSLC formalisms
PyDynAA	Entity · Event · EventHandler · SimulationEngine (discrete-event engine)



The Component



Every physical device in the network is a Component. A component is composed of:

Properties

Physical values: channel length, number of memory slots, emission period.

Models

Describe physical behavior: FibreDelayModel, DepolarNoiseModel, FibreLossModel.

Ports

Communication interfaces. Ports can be connected, forwarded, and send both quantum and classical messages.

Subcomponents

Nested components. A Node contains a Processor; a Network contains Nodes.

Example: `Alice.subcomponents['alice_source'].ports['qout0'].forward_output(alice.ports[qout])`

Quantum State Representations in NetSquid

Code	Full name	Mixed states	Memory	Best for
KET	Ket vector	No	$O(2^n)$	Small pure-state simulations
DM	Density matrix	YES ✓	$O(2^{2n})$	Noisy simulations (recommended)
STAB	Stabilizer tableau	No	$O(n^2)$	Large Clifford-only circuits
GSLC	Graph state + Cliffords	No	$O(nd)$	Sparse entanglement, 1000+ nodes

Rule of thumb: Few qubits + noise → DM. Many qubits + Clifford gates → STAB or GSLC.

Set with: `ns.set_qstate_formalism(ns.QFormalism.DM)`

The Protocol

A Protocol is an asynchronous coroutine that runs on a node and reacts to events:

```
class MyProtocol(NodeProtocol):
    def run(self):
        while True:
            yield self.await_port_input(self.node.ports['qin']) # suspend until qubit arrives
            yield self.await_program(self.node.qmemory)         # suspend until program done
            yield self.await_timer(50)                           # sleep 50 ns
            self.node.ports['cout'].tx_output({'result': 42})     # send classical msg
```

`await_port_input(port)`

Suspend until a message arrives on the port.

`await_program(memory)`

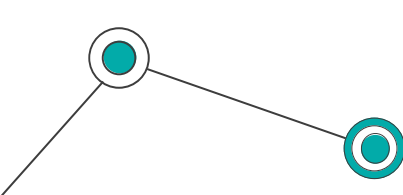
Suspend until a quantum program finishes executing.

`await_timer(duration)`

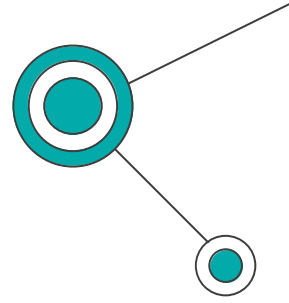
Sleep for a fixed simulated time duration.

`tx_output(msg)`

Send a classical or quantum message through a port.



Physical Models



DELAY MODELS

FibreDelayModel — propagation delay from fiber length & speed of light in glass ($n=1.44$)

FixedDelayModel — constant delay (e.g., source emission period)

NOISE MODELS

DepolarNoiseModel — random X/Y/Z errors with rate p per nanosecond

T_1/T_2 dephasing — applied retroactively when qubit is accessed after idle time Δt

LOSS MODELS

FibreLossModel($p_{\text{loss_init}}$, $p_{\text{loss_length}}$) — coupling loss at source + per-km attenuation

Qubit encoded in photon that is lost → qubit is discarded from the simulation

Models are assigned to components and called automatically when a qubit is accessed or transmitted — no manual accounting needed.

Installing NetSquid

Step 1 – Register at netsquid.org (free account required for pip access)

```
# Step 2 – Install from the NetSquid private index (use your registered credentials)
pip install netsquid --extra-index-url https://pypi.netsquid.org
```

```
# Or set credentials as environment variables:
```

```
export NETSQUIDPYPI_USER=your_username
```

```
export NETSQUIDPYPI_PWD=your_password
```

```
pip install netsquid --extra-index-url https://${NETSQUIDPYPI_USER}:${NETSQUIDPYPI_PWD}@pypi.netsquid.org
```

```
# Step 3 – Verify installation
```

```
import netsquid as ns
```

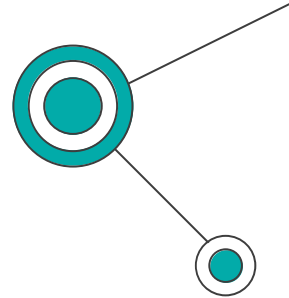
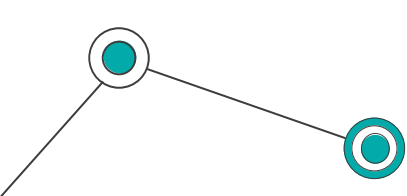
```
print(ns.__version__)          # e.g. 1.1.6
```

```
# Step 4 – Set quantum state formalism
```

```
ns.set_qstate_formalism(ns.QFormalism.DM)  # density matrix (handles noise)
```

```
ns.set_qstate_formalism(ns.QFormalism.KET)  # ket vector (faster, no mixed states)
```

Requirements: Python 3.6+ · numpy · scipy · pydynaa (auto-installed) | Docs: <https://docs.netsquid.org>

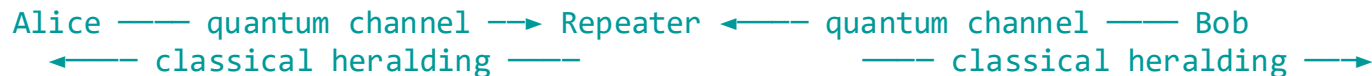


PART III

Code Walkthrough

A complete 3-node repeater simulation in NetSquid

The Scenario We're Simulating



- 1 Alice and Bob each trigger their local Bell source (`QSource.trigger()`)
- 2 One qubit of each pair travels to the Repeater via quantum fiber channel
- 3 The other qubit stays in local quantum memory (slot 0)
- 4 Repeater performs a Bell State Measurement on the two arriving qubits
- 5 Repeater sends BSM result (heralding dict) to both Alice and Bob
- 6 Alice and Bob measure their local qubits in Z or X basis

Physics: After the BSM, A_{Local} and B_{Local} are entangled — this is entanglement swapping!

Building Blocks

```
def build_processor(name, num_positions):
    phys_instr = [
        PhysicalInstruction(INSTR_X,          duration=1, parallel=True),
        PhysicalInstruction(INSTR_CNOT,       duration=4, parallel=False),
        PhysicalInstruction(INSTR_MEASURE,    duration=5, parallel=False),
        PhysicalInstruction(INSTR_MEASURE_BELL, duration=8, parallel=False),
    ]
    return QuantumProcessor(name, num_positions=num_positions, phys_instructions=phys_instr)

def build_bell_source(name, period=20):
    return QSource(name,
        state_sampler = StateSampler([b00], [1.0]), # always emits |0+⟩
        num_ports = 2, # one port per qubit
        status = SourceStatus.EXTERNAL, # triggered manually
        models = {"emission_delay_model": FixedDelayModel(delay=period)})
```

Memory slots: Alice = 1 slot · Bob = 1 slot · Repeater = 2 slots (one per incoming qubit)

QSource ports: qout0 → travels to Repeater via quantum channel | qout1 → stored locally

fallback_to_nonphysical=True allows non-physical operations for prototyping — a handy option for demos.

Channels & Port Wiring

```
# Quantum channel: fiber delay + optional depolarizing noise + optional loss
channel = QuantumChannel("q_alice_rep", length=10,
    models={"delay_model": FibreDelayModel(),
        "quantum_noise_model": DepolarNoiseModel(depolar_rate=0.0), # ideal demo
        "quantum_loss_model": FibreLossModel(p_loss_init=0.0, p_loss_length=0.0)})

# Classical channel: fiber delay only
classical = ClassicalChannel("c_rep_alice", length=10,
    models={"delay_model": FibreDelayModel()})
```

```
# Wire Alice source: split the Bell pair across network and local memory
alice_source.ports["qout0"].forward_output(alice.ports[alice_qout]) # → Repeater
alice_source.ports["qout1"].connect(alice.qmemory.ports["qin0"])    # stays local

# Wire Repeater inputs: each arriving qubit has its own memory slot
repeater.ports[rep_qin_from_alice].forward_input(repeater.qmemory.ports["qin0"])
repeater.ports[rep_qin_from_bob].forward_input(repeater.qmemory.ports["qin1"])
```

To study noise: set $\text{depolar_rate} > 0$ or $\text{p_loss_length} > 0$ — this is what the paper does for NV-center chains.

The Bell Measurement Program

```
class BellMeasureProgram(QuantumProgram):
    default_num_qubits = 2

    def program(self):
        q1, q2 = self.get_qubit_indices(2)
        self.apply(INSTR_MEASURE_BELL, [q1, q2], output_key="bell_index")
        yield self.run() # duration = 8 ns (physical instruction)

# Used in the Repeater protocol:
prog = BellMeasureProgram()
self.node.qmemory.execute_program(prog, qubit_mapping=[0, 1])
yield self.await_program(self.node.qmemory)
bell_index = prog.output["bell_index"][0] # 0,1,2,3 → Φ+, Φ-, Ψ+, Ψ-
```

The 4 possible Bell states measured:

0 → $|\Phi^+\rangle$

$$\frac{(|00\rangle + |11\rangle)}{\sqrt{2}}$$

1 → $|\Phi^-\rangle$

$$\frac{(|00\rangle - |11\rangle)}{\sqrt{2}}$$

2 → $|\Psi^+\rangle$

$$\frac{(|01\rangle + |10\rangle)}{\sqrt{2}}$$

3 → $|\Psi^-\rangle$

$$\frac{(|01\rangle - |10\rangle)}{\sqrt{2}}$$

In a full protocol, alice & bob use bell_index to apply correction gates. Here we log it for analysis.

Alice/Bob Protocol — SourceNodeProtocol

```
def run(self):
    for r in range(1, self.rounds + 1):
        qsource.trigger()          # 1. emit Bell pair
        yield self.await_port_input(qin0)  # 2. wait for local qubit in memory

        yield self.await_port_input(cin)    # 3. wait for heralding from Repeater
        herald = cin.rx_input().items[0]    # 4. read BSM result

        q = self.node.qmemory.pop(positions=[0])[0]  # 5. retrieve local qubit
        obs = ns.Z if self.round_id % 2 == 0 else ns.X  # alternate basis Z/X
        outcome, _ = ns.qubits.measure(q, observable=obs)  # 6. measure

        yield self.await_timer(self.interval)  # 7. pause before next round
```

Basis choice: Z on even rounds, X on odd rounds. Deterministic (not random like BB84) — this is a demo.

Heralding wait: Protocol blocks at step 3 until the Repeater sends a classical message — after BSM completes and propagates over 10 km fiber.

Repeater Protocol

```
def run(self):
    while True:
        # 1. Wait for BOTH qubits to arrive (concurrent AND condition)
        yield self.await_port_input(port_a) & self.await_port_input(port_b)

        # 2. Verify memory slots are populated
        qa = self.node.qmemory.peek(positions=[0])[0]
        qb = self.node.qmemory.peek(positions=[1])[0]

        # 3. Execute Bell State Measurement
        prog = BellMeasureProgram()
        self.node.qmemory.execute_program(prog, qubit_mapping=[0, 1])
        yield self.await_program(self.node.qmemory)

        # 4. Send heralding to Alice AND Bob
        herald = {"type": "bsm_result", "bell_index": str(bell_index), "time": ns.sim_time()}
        cout_alice.tx_output(herald)
        cout_bob.tx_output(herald)

        # 5. Clean memory for next round
        for pos in [0, 1]:
            self.node.qmemory.pop(positions=[pos])
```

AND operator: `await port_a & await port_b` blocks until BOTH events occur. Same syntax as Python bitwise AND — special event expression in PyDynAA.

Running the Simulation – main()

```
def main():
    ns.sim_reset()                # clean slate
    ns.set_qstate_formalism(ns.QFormalism.DM) # density matrix: handles noise

    network, protocols = build_network() # build all nodes, channels, sources
    print_topology(network)             # debug: print all ports & slots

    for p in protocols:            # start all 3 protocols
        p.start()                 # Alice protocol, Bob protocol, Repeater protocol

    stats = ns.sim_run(duration=700_000) # run for 700 ms simulated time
    print(stats)                    # event counts, qubit ops, state sizes
```

Why DM formalism?

More general than KET — handles mixed states and noise correctly. Falls back to KET if DM is unavailable.

Why duration 700,000?

6 rounds with 50 ns interval each uses little time. Large duration ensures the Repeater's while True loop doesn't terminate early.

What stats contains?

Event counts processed, quantum operation counts, max & avg state sizes across the simulation.

What This Code Is and What It Is Not

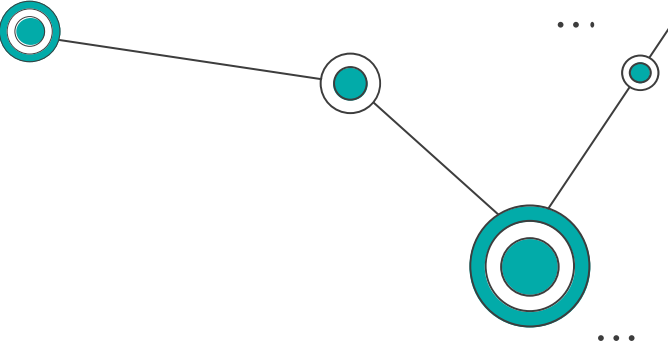
WHAT IT DEMONSTRATES

- ✓ Build a 3-node quantum network in NetSquid
- ✓ Wire a QSource across network and local memory
- ✓ Implement concurrent protocols with await & event expressions
- ✓ Perform a Bell State Measurement in a quantum program
- ✓ Coordinate quantum & classical information with heralding

INTENTIONALLY OMITTED

- X Correction gates based on bell_index
- X QBER (Quantum Bit Error Rate) measurement
- X Sifting — same-basis comparison only
- X Key generation & privacy amplification
- X Realistic noise and loss parameters
- X Statistical averaging over many runs

This is an architectural and pedagogical template, the scaffolding of a real quantum network protocol.



KEY TAKEAWAYS

- ① A quantum network needs 5 elements: source, quantum channel, quantum memory, processor, classical channel. Each has noise, delay, and failure modes.
- ② NetSquid models all of this with discrete-event simulation — accurate timing, physical noise models, modular components, scalable protocols.
- ③ Entanglement swapping is the core operation that lets quantum networks extend beyond direct fiber limits — and the code we studied implements exactly this.

